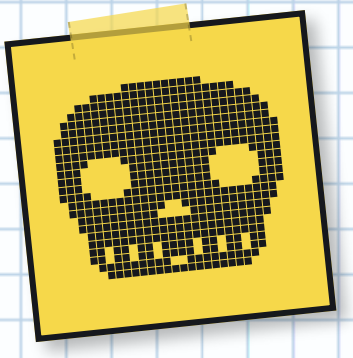
HELLO
my name is

JARVIS

esp32 · wroom-32



· · · ONE NIGHT ONLY · · ·

ADMIT ONE

192.168.4.1 · port 80

★ ★ ★ ★ ★

NO. 0001

FROM THE BENCH

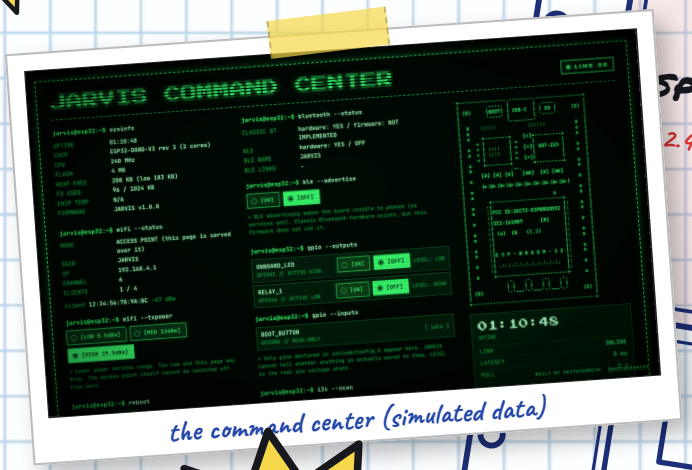
- ESP32 DevKit (WROOM-32)
- USB data cable
- laptop + Chrome
- VS Code + PlatformIO
- optional: LED + 330 Ω

no router · no cloud · no internet

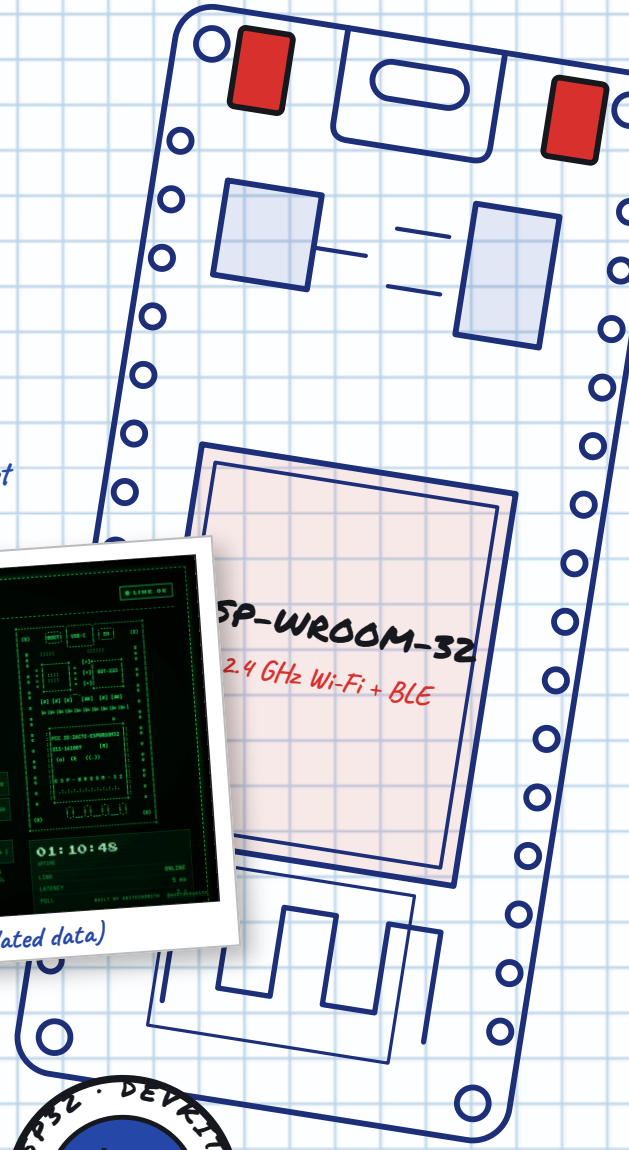
JARVIS

ESP32 COMMAND CENTER

just a rather very intelligent system — a desktop-style dashboard your ESP32 serves from its own Wi-Fi.



the command center (simulated data)



BOOT!

LOGIN → ACCESS GRANTED → BOOT → COMMAND CENTER

PlatformIO · Arduino · C++ · LittleFS · ESPAsyncWebServer · HTML / CSS / JS

BUILT BY ADITECHSMITH
@aditechsmith

Contents

1	Overview	2
2	Key features	3
3	Verification status	3
4	Interface preview	4
5	System architecture	5
6-8	Hardware · software · components	6
9	Project structure	6
10-11	How it works · firmware	7
12-14	Frontend · LittleFS · PlatformIO	8
15-18	Setup, build, upload (Windows)	9
19-20	Connect · configuration	10
21	Troubleshooting	11
22-23	Security · limits · ideas	12
24-26	Contributing · license · credits	13

tip: every step is written for a beginner →

1 Overview

JARVIS turns an **ESP32-WROOM-32** development board into a self-contained, desktop-style command center. The board creates its own Wi-Fi access point, serves a single-page web app from flash memory, and lets a logged-in user watch device status and control a small, explicitly configured set of hardware features.

There is **no internet, router or cloud** involved: join the ESP32's Wi-Fi from a laptop and open `http://192.168.4.1/` in Chrome. The interface is **desktop-first** — it targets a laptop browser, not a phone.

LOGIN → ACCESS GRANTED → ESP32 BOOT → COMMAND CENTER

Why this project?

- Learn how an ESP32 can host a real web app from LittleFS.
- Practise safe hardware control: only pins *you* list can be driven.
- Explore retro-terminal UI design (phosphor-green, pixel fonts, CRT scan-lines) that works fully offline.
- Portfolio-ready: documented, honest about what is tested and what is not.

Read this first

Pages marked `UNTESTED` describe features that compile in theory but have **not** been verified on real hardware. Test them on your board and report what you find.

2 Key features

Area	What it does
Wi-Fi access point	The board hosts its own WPA2 network; SSID and password are yours to set.
Login	Credentials are checked **in the firmware**, not in browser JavaScript.
Sessions	128-bit random token in an HttpOnly, SameSite=Strict cookie; 30 min idle timeout; max 3 sessions.
Brute-force limit	Per-IP lockout after 5 wrong tries: 60 s, doubling up to ×8.
Boot animation	An SVG ESP32 board draws itself while a log shows real values read from the device.
System info	Uptime, chip model / revision, cores, CPU MHz, flash, free heap, filesystem use, firmware version.
Wi-Fi panel	SSID, IP, channel, connected clients (MAC + RSSI), TX power LOW / MED / HIGH.
Bluetooth	BLE advertising on / off with connection count. Classic Bluetooth is **not** implemented — the UI says so.
GPIO	Outputs and inputs only for pins listed in `include/config.h`; unsafe pins show BLOCKED.
I2C scanner	Scans SDA 21 / SCL 22 and lists addresses with "possible device" hints.
System control	Two-step reboot and logout.
Offline UI	All four fonts are served from the board; nothing loads from the internet.

3 Verification status

Item	Status
Frontend: 4 screens, login errors, lockout, animations, controls	**VERIFIED** vs. mock API
Font files match `@font-face` paths and load	**VERIFIED** in Chrome
Firmware compiles and flashes	**REPORTED** by author
Wi-Fi AP + web server answer on 192.168.4.1	**REPORTED** by author
Full UI served from LittleFS on hardware	**TO CONFIRM**
GPIO / BLE / I2C / TX power / reboot on hardware	**UNTESTED**
Stability with several clients, BLE + Wi-Fi memory	**UNTESTED**

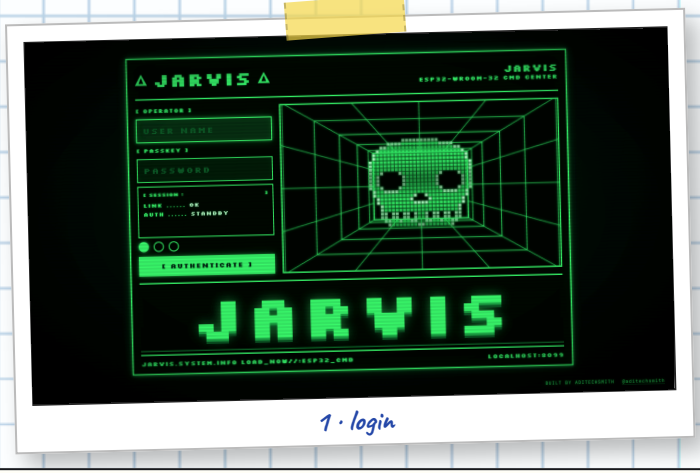
"reported" = the author flashed a real board and reached the page; nothing here was captured from hardware by the documentation tools.

Honest scope

- Hobby-grade security — **not** production.
- HTTP only; WPA2 on the AP is the real protection.
- BLE = advertising only. No OTA updates.
- No hardware capability is claimed beyond what the source code implements.

4 Interface preview

four screens, one flow



1 · login

Login

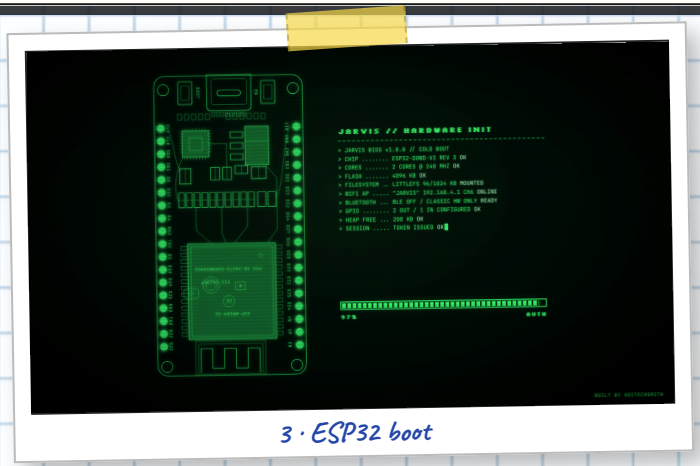
Pixel skull in a wire tunnel, operator / passkey fields and a session log. Wrong password → ACCESS DENIED + tries left; five misses → locked countdown.



2 · access granted

Access granted

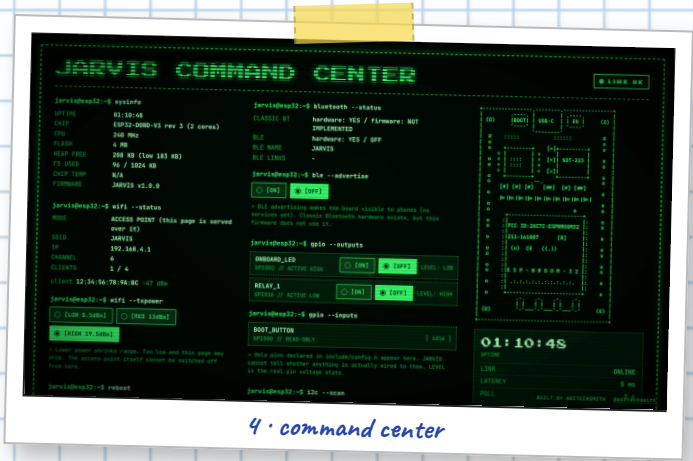
Glowing grid and a flicker-in ACCESS GRANTED, shown while the page fetches real device data.



3 · ESP32 boot

ESP32 boot

The board is drawn line by line; the log prints the real chip, flash, filesystem, SSID and heap values.



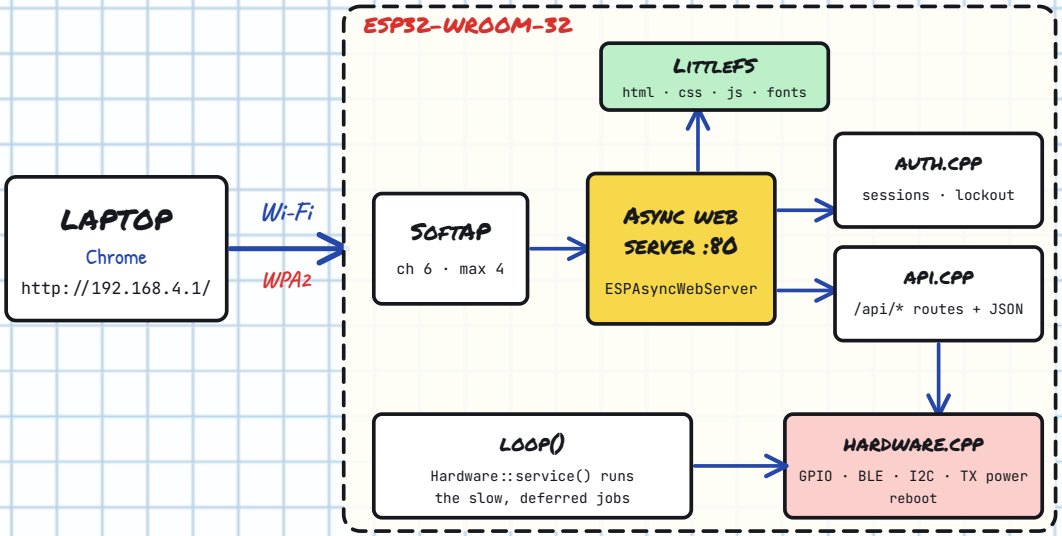
4 · command center

Command center

Terminal panels: sysinfo, wifi, bluetooth, gpio, i2c, reboot / logout, ASCII board, live uptime. Polls /api/status every 2 s.

Screenshots come from the UI talking to a simulated API: sample data, a fake MAC and an extra example RELAY_1 row — not hardware captures. Keep real credentials and network details out of your own screenshots.

5 System Architecture



Login Flow

1. `GET /` loads the page, `GET /api/session` says `authenticated:false`.
2. `POST /api/login` with the form fields.
3. Firmware answers **200** + cookie, **401** (+ tries left) or **429** (+ retry seconds).
4. On 200 the page reads `/api/info` and `/api/status` and plays ACCESS GRANTED → boot → dashboard.

Slow work (BLE start/stop, I2C scan, reboot) is only requested in the web handler and executed from `loop()`, so the web-server task never blocks.

Public (no session)

- Static files: `index.html`, `style.css`, `script.js`, fonts
- `GET /api/session`
- `POST /api/login` · `/api/logout`

Needs a session cookie

- `/api/info` · `/api/status`
- `/api/gpio` · `/api/bluetooth`
- `/api/wifi/txpower` · `/api/i2c/scan` · `/api/reboot`
- POSTs also check `Origin`

Deferred to loop()

- BLE start / stop
- I2C scan
- Reboot (after ≈0.8 s)
- Result is read back via `/api/status`

6 Hardware

ESP32-WROOM-32 dev board	PlatformIO board `esp32dev`. S2 / S3 / C3 are **not** targeted.
USB data cable	A charge-only cable cannot flash.
Windows laptop + Wi-Fi	The client; Chrome recommended.
Optional LED + 220–330 Ω	Test an extra output pin.
Optional I2C device	Test the scanner on GPIO 21 / 22.

7 Software

VS Code	Editor.
PlatformIO IDE extension	Build, upload, serial monitor.
CP210x / CH340 driver	Depends on the board's USB chip.
Google Chrome	The browser used for testing.
Libraries	Fetched automatically on first build.

8 Components

Component	Pin	Mode
ONBOARD_LED	GPIO 2	output, active-HIGH
BOOT_BUTTON	GPIO 0	input, pull-up, active-LOW
I2C bus	21 / 22	scan only
BLE radio	—	advertising “JARVIS”
Wi-Fi radio	—	SoftAP, ch 6, 4 clients

JARVIS can't see what is wired to a pin — wiring a relay correctly is on you.

9 Project structure

```
ESP32_Command_Center/
├─ platformio.ini        board · libs · partitions
├─ include/
│  ├─ config.h           pins, limits  (edit me)
│  ├─ secrets.example.h  placeholders (committed)
│  └─ secrets.h          YOUR creds (git-ignored)
├─ src/
│  ├─ main.cpp           LittleFS → AP → server
│  ├─ api.h / api.cpp    HTTP routes + JSON
│  ├─ auth.h / auth.cpp  sessions, lockout
│  └─ hardware.h / .cpp  GPIO · BLE · I2C · radio
├─ data/                 → LittleFS image
│  ├─ index.html         four screens
│  ├─ style.css          green terminal theme
│  ├─ script.js          logic + animations
│  └─ fonts/             4 × .woff2 (SIL OFL)
├─ docs/  SETUP.md · TROUBLESHOOTING.md · images/
├─ CONTRIBUTING.md   SECURITY.md   .gitignore
└─ README.md
```

Two things get uploaded

The **firmware** (`src/` + `include/`) and the **filesystem image** (`data/`) are separate uploads. Change `config.h` / `secrets.h` → upload firmware. Change anything in `data/` → upload filesystem.

10 How it works

1. **Boot.** `setup()` mounts LittleFS, initialises GPIO and auth state, starts the SoftAP, registers routes and starts the server on port 80. `loop()` only calls `Hardware::service()`.
2. **Static files.** `serveStatic("/", LittleFS, "/")` serves the frontend; `/fonts/*` has its own handler so `.woff2` gets the right content type.
3. **Login.** The browser POSTs the form to `/api/login`; the firmware compares in constant time against `secrets.h`.
4. **Everything else needs a session.** Data and control routes call `Auth::check()`; POSTs also check `Origin == Host`.
5. **Slow actions are deferred** to `loop()`; the dashboard polls `/api/status` every 2 s for the result.

11 Firmware

main.cpp	Serial banner, LittleFS mount (no auto-format), SoftAP, TX power, server start.
api.cpp	All routes, hand-built JSON (no ArduinoJson), static files, 404 handler.
auth.cpp	RAM session table, `esp_random()` tokens, constant-time compare, per-IP lockout.
hardware.cpp	GPIO with safe-pin allow-list, BLE, I2C scan, TX power, reboot, chip temperature.
config.h	Pins, limits and timings you will edit.
secrets.h	AP SSID / password + dashboard login (never committed).

HTTP API

Route	Auth	Purpose
GET /api/session	no	{authenticated}
POST /api/login	no	user, pass → 200+cookie / 401 / 429
POST /api/logout	no	clears the caller's session
GET /api/info	yes	chip, cores, flash, fs size
GET /api/status	yes	uptime, heap, wifi, BT, gpio, i2c
POST /api/gpio	yes	id, state — configured outputs only
POST /api/bluetooth	yes	state 0/1 (applied in loop)
POST /api/wifi/txpower	yes	level 0 / 1 / 2
POST /api/i2c/scan	yes	start scan; read result in status
POST /api/reboot	yes	restart after ≈0.8 s

Tuning knobs (config.h)

AP channel / clients	6 / max 4
Session idle timeout	30 min · max 3 sessions
Login lockout	5 fails → 60 s (×2 … ×8)
I2C pins	SDA 21 · SCL 22
BLE name	JARVIS

"yes" means a valid `JARVIS_SID` session cookie is required.

12 Frontend

index.html	Four `<section>` screens: login, access granted, boot, dashboard — plus the small fixed creator credit.
style.css	Phosphor-green on near-black, pixel fonts, CRT scan-lines, desktop-only layout (`min-width:1100px`). `@font-face` → `fonts/*.woff2`.
script.js	One IIFE: `api()` wrapper, screen manager, login canvas effects, boot timeline, SVG board, 2 s polling, buttons with loading / error states. Nothing is stored in browser storage.

13 LittleFS

- Files in `data/` are packed into a LittleFS image and flashed **separately** from the firmware.
- `board_build.filesystem = littlefs` ; `huge_app.csv` leaves ≈3 MB for the app and ≈1 MB for files.
- Mounted with `LittleFS.begin(false)` — it **never auto-formats**. A failed mount means the image was not uploaded.
- Font names are case-sensitive: `jbmono-400`, `jbmono-700`, `pressstart2p`, `silkscreen-bold` (.woff2).
- Re-upload the filesystem image whenever `data/` changes.

14 PlatformIO

```
; platformio.ini
[env:esp32dev]
platform  = espressif32
board     = esp32dev
framework = arduino

monitor_speed = 115200
upload_speed  = 460800

board_build.filesystem = littlefs
board_build.partitions = huge_app.csv
; Wi-Fi + BLE needs the big app slot (no OTA)

lib_deps =
  esp32async/AsyncTCP@^3.5.0
  esp32async/ESPAsyncWebServer@^3.7.0

build_flags =
  -D CORE_DEBUG_LEVEL=1
```

Heads-up

`platform` is **not** version-pinned. When you have a build that works, pin it (`espressif32@<version>`) for reproducible builds. No ArduinoJson: requests are form-encoded and responses are small hand-built JSON.

15 Setup on Windows

1. **Install** VS Code, then the **PlatformIO IDE** extension. Install the USB driver (CP210x / CH340) if Windows shows no COM port.
2. **Open the project**: *File → Open Folder*, pick the folder that contains `platformio.ini`.
3. **Create your private credentials file.** The repo ships only a template:

```
:: Windows (cmd / PlatformIO terminal)
copy include\secrets.example.h include\secrets.h
```

```
// include/secrets.h  — your own values
#define JARVIS_AP_SSID       "YOUR_WIFI_SSID"
#define JARVIS_AP_PASSWORD   "YOUR_WIFI_PASSWORD"   // 8-63 chars
#define JARVIS_LOGIN_USER    "YOUR_LOGIN_USER"
#define JARVIS_LOGIN_PASS    "YOUR_LOGIN_PASSWORD"
```

Never commit `secrets.h`. It is in `.gitignore`. Use long, unique passwords — the placeholders are not safe to keep.

16-18 Build

4. **Build the firmware** *(expect SUCCESS)*

```
pio run
```

5. **Flash the firmware.** If it hangs at `Connecting....`, hold the **BOOT** button.

```
pio run -t upload
```

6. **Build + upload the filesystem image** (do not skip!). Close the Serial Monitor first.

```
pio run -t buildfs
pio run -t uploadfs
```

7. **Press EN** (reset) on the board, then open the monitor:

```
pio device monitor
```

Same thing in the sidebar: Project Tasks → esp32dev → General → Build / Upload, and Platform → Build / Upload Filesystem Image.

NOTE A `secrets.h: No such file` error means step 3 was skipped. The first build downloads the toolchain and libraries — it can take several minutes.

19 Connect & Open

```
[serial monitor, 115200 baud]
[FS] LittleFS mounted (... bytes used)
  index.html · style.css · script.js · fonts/
[GPIO] output ONBOARD_LED  GPIO2  ready
[GPIO] input  BOOT_BUTTON  GPIO0  ready
[WIFI] AP "<your SSID>" up on channel 6
[WIFI] open http://192.168.4.1/
[HTTP] server started on port 80
[SYS] READY
```

1. Join the Wi-Fi network named in `secrets.h` (Windows may say "no internet" — stay connected).
2. Open `http://192.168.4.1/` in Chrome — type `http://` ; the board does not serve HTTPS.
3. Log in with **your own** username / password from `secrets.h` .

`192.168.4.1` is the ESP32 SoftAP's default address — not a private address of yours.

UNTESTED

optional hardware checks (LED, BOOT button, BLE, I2C, reboot) are in docs/SETUP.md

20 Configuration

What	Where
AP name / password, login	`include/secrets.h`
Channel, clients, hidden SSID	`config.h` (JARVIS_AP_*)
Timeout, sessions, lockout	`config.h`
I2C pins	`config.h` I2C_SDA / SCL
Add a controllable output	row in `OUTPUTS[]`
Add a read-only input	row in `INPUTS[]`
Colours and layout	`data/style.css` → upload FS

Safe output pins

`2 4 5 13 14 15 16 17 18 19 23 25 26 27 32 33` — enforced at boot. Flash pins, UART 0, strapping pins, input-only 34-39 and the I2C pins are refused and shown as BLOCKED.

```
// external LED: GPIO4 → 220-330Ω → LED → GND
{ 4, "LED_EXT_1", true, false },
//  pin, name,       activeHigh, initialOn
```

config.h / secrets.h changed → upload firmware. **data/** changed → upload filesystem.

21 Troubleshooting

check the serial monitor first — redact before sharing!

Symptom	Likely cause	Fix
secrets.h: No such file	Credentials file not created	`copy include\secrets.example.h include\secrets.h`, then edit.
Build error: ESPAsyncWebServer / AsyncTCP	Stale or duplicate library copies	Run Clean, delete `.pio\libdeps`, build again; nothing extra in `lib\`.
"Failed to connect to ESP32"	Not in download mode, bad cable, port busy	Hold BOOT while uploading; use a data cable; close the serial monitor.
No COM port in Windows	Missing USB-serial driver	Install the CP210x or CH340 driver for your board.
Page says "filesystem not mounted"	Filesystem image not uploaded or mount failed	Close the monitor → Build FS Image → Upload FS Image → press EN. Serial must show `[FS] LittleFS mounted`.
Still MOUNT FAILED after FS upload	`data/` not in project root, or flash layout changed	`data/` must sit next to `platformio.ini`. Run Erase Flash → upload firmware → upload FS.
Page unstyled / wrong fonts	Old image or renamed font files	Check the 4 lowercase names in `data/fonts/`; re-upload FS; hard-refresh (Ctrl+F5).
Can't see the Wi-Fi network	AP failed to start	Look for `softAP FAILED`; password must be 8–63 chars; reset the board.
Browser can't reach the page	Wrong network or https://	Join the ESP32 network (not mobile data) and type `http://192.168.4.1/`.
Login always ACCESS DENIED	Different credentials than the flashed ones	Re-check `secrets.h`; rebuild; upload **firmware** again.
Button shows LOCKED	Lockout working as designed	Wait for the countdown, or reset the board.
Back at login after reboot	Sessions are RAM-only	Expected — log in again.
Brownout / reset loop	Weak USB port or cable	Another port / cable / powered hub.
Random resets with BLE on	Memory pressure	Turn BLE off; watch free heap.
Chip temp N/A · output BLOCKED	Not supported · pin not on allow-list	Normal · use one of the safe pins.

Still stuck? Open an issue with board model, Windows version, the redacted serial log and the exact error — never real passwords or personal paths.

22 Security

Implemented

- Credentials verified in firmware; the browser never holds them.
- Constant-time comparison; user *and* password always evaluated.
- 128-bit hardware-RNG session tokens; HttpOnly + SameSite=Strict cookie.
- Per-IP lockout; 30 min idle expiry; limited session count.
- Every data / control route needs a session; POSTs check `Origin`.
- Only pins in `config.h` *and* on the safe-pin list can be driven.
- No credentials in the repo; `secrets.h` is git-ignored.

Honest limitations

- **Plain HTTP.** Someone who captures the Wi-Fi traffic *and* knows the AP password could read the cookie. Choose a strong AP password.
- Credentials are compiled into the firmware — no hashing, no change-password screen; flash readers can extract them.
- Sessions vanish on reboot; lockout is per IP, so a changing IP can keep guessing slowly.
- The HTML / CSS / JS are public to anyone on the network; only data and controls are protected.
- **Hobby-grade — not production security.**

23 Limits & Ideas

Known limitations

- Desktop layout only — no phone breakpoints.
- Classic Bluetooth unused; BLE = advertising only.
- No OTA updates (`huge_app.csv` has no OTA slots).
- Chip temperature may read `N/A`.
- Hardware actions not yet verified on boards.

Ideas — not implemented

- HTTPS, or a change-password flow stored in NVS.
- A BLE GATT service; sensor modules for the empty SLOT_A / SLOT_B.
- Pin `platform` and library versions after hardware checks.
- Compile-time guard that refuses to build with placeholder credentials.

Privacy checklist before you publish

- `include/secrets.h` is **not** in the repo.
- Screenshots show no real SSID, login or MAC.
- No personal paths (`C:\Users\…`) in logs or issues.
- Passwords were changed from the placeholders.

Passwords once pushed live in Git history — if that happens, rotate them.

24 Contributing

- Open an issue first for anything bigger than a small fix.
- Build with `pio run`; re-upload the FS after changing `data/`.
- New hardware goes through `config.h` and the safe-pin allow-list.
- Keep slow work out of the web-server task — queue it for `loop()`.
- Label anything you could not test on hardware as untested.
- **Never** post real Wi-Fi details, passwords, tokens or personal paths in code, issues, logs or screenshots.

25 License

No license has been selected yet. Until the owner adds a `LICENSE` file, the default is "all rights reserved" and others may not legally reuse the code. **The owner must choose.**

Suggestion, not a decision: the **MIT License** — short, permissive, common for hobby firmware. Alternatives: Apache-2.0 (explicit patent grant), GPL-3.0 (derivatives must stay open).

Third-party parts keep their own licenses: the fonts (Press Start 2P, Silkscreen, JetBrains Mono) are SIL Open Font License 1.1; AsyncTCP and ESPAsyncWebServer are separate libraries — check their repos before redistributing binaries.

26 Credits

PROJECT CREDIT

BUILT BY ADITECHSMITH

Instagram · @aditechsmith

https://www.instagram.com/aditechsmith?obrf=MXZlOWpub2k2M3RrbQ==

Built with PlatformIO, the Arduino core for ESP32 and the ESP32Async libraries. Fonts: Press Start 2P, Silkscreen and JetBrains Mono (SIL OFL). This PDF's notebook look uses Permanent Marker, Caveat, Patrick Hand and Special Elite (SIL OFL), with original artwork.

THANKS FOR BUILDING ★